

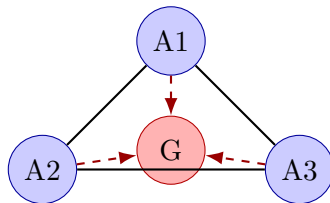
SORBONNE UNIVERSITÉ

MASTER AI2D – M1

UE FoSyMA – FONDEMENTS DES SYSTÈMES MULTI-AGENTS

Rapport de Projet

Système Multi-Agents pour l'Exploration Coopérative
et la Chasse de Golems sur la Plateforme Dedale/JADE



Groupe : 9

Membres : Riadh Ouazib, Radouane Aouini

Dépôt Git : <https://github.com/Radouane-ANI/dedale>

Date : 29 avril 2026

Plateforme : JADE + Dedale (GraphStream)

Table des matières

1	Introduction	2
1.1	Objectifs du projet	2
1.2	Contexte et plateforme	2
2	Architecture générale du code	2
2.1	Structure du projet	2
2.2	Diagramme des composants par agent	3
2.3	Diagramme d'états du comportement principal	4
2.4	Diagramme d'états du protocole de partage	4
2.5	Explication des interactions principales	5
3	Choix d'algorithmes et stratégies retenues	6
3.1	Exploration	6
3.2	Communication	6
3.3	Coordination	7
3.4	Chasse	7
4	Analyse des algorithmes	8
4.1	Algorithme d'exploration	8
4.2	Algorithme de communication	9
4.3	Algorithme de coordination	9
4.4	Algorithme de chasse	10
5	Conclusion	11
5.1	Synthèse des résultats	11
5.2	Regard critique	11
5.3	Extensions possibles	11

1 Introduction

1.1 Objectifs du projet

Ce projet a pour objectif la conception et l'implémentation d'un système multi-agents (SMA) capable de résoudre, de manière coopérative et décentralisée, deux problèmes étroitement liés :

1. **L'exploration collaborative** d'un environnement représenté par un graphe non orienté inconnu *a priori*. Chaque agent doit contribuer à découvrir l'intégralité de la topologie tout en évitant la redondance avec ses pairs.
2. **La chasse coordonnée** (et l'éventuel encerclement) d'agents adverses appelés *Golems*. Ces entités émettent une signature olfactive (*Stench*) sur les nœuds adjacents, ce qui constitue l'unique source d'information indirecte permettant aux explorateurs de déduire leur localisation.

L'objectif transverse est de produire un comportement opportuniste, capable de *basculer dynamiquement* entre exploration et poursuite en fonction des stimuli observés (visibilité directe d'un Golem, fraîcheur d'une trace olfactive, signal de siège partagé), tout en assurant une coordination sans serveur centralisé.

1.2 Contexte et plateforme

JADE. La plateforme *Java Agent DEvelopment framework* (JADE) fournit un conteneur d'agents conforme aux spécifications FIPA. Chaque agent est un objet Java actif disposant d'une boîte aux lettres ACL et d'une liste de *behaviours* ordonnancés coopérativement. Les protocoles de communication s'appuient sur les performatives FIPA-ACL standards (INFORM, REQUEST, etc.).

Dedale. Dedale est un environnement de simulation construit au-dessus de JADE et de la bibliothèque GraphStream. Il fournit :

- une carte sous forme de graphe instancié à partir d'un fichier `.dgs` ;
- une API d'observation locale (`observe()`) qui retourne, pour le nœud courant et ses voisins immédiats, les éléments présents : agents (`AGENTNAME`), trésors, stenchs, etc. ;
- une API de déplacement (`moveTo()`) limitée aux nœuds adjacents au nœud courant.

Agents explorateurs. Les explorateurs sont des `ExploreCoopAgent` héritant de `AbstractDedaleAgent`. Chacun maintient localement une représentation *partielle* du graphe (`MapRepresentation`) qu'il complète au fil des observations et qu'il fusionne avec celles de ses coéquipiers via un protocole de partage delta.

Golems. Les Golems sont des agents adversaires se déplaçant au sein du graphe. Leur seule trace exploitable est l'odeur (`STENCH`) laissée sur les nœuds voisins de leur position courante. Ils peuvent également être directement observés lorsqu'ils se trouvent dans le voisinage visuel d'un explorateur.

2 Architecture générale du code

2.1 Structure du projet

L'arborescence pertinente du projet est la suivante :

```

dedale-etu/
|-- pom.xml
|-- resources/                                # Cartes .dgs (topologies de test)
|-- src/main/java/eu/su/mas/dedaleEtu/
|   |-- mas/
|   |   |-- agents/
|   |   |   '-- dummies/
|   |   |       '-- explo/
|   |   |           |-- ExploreCoopAgent.java      # Agent principal
|   |   |           '-- ExploreSoloAgent.java
|   |   |-- behaviours/
|   |   |   |-- OpportunisticBehaviour.java        # Coeur decisionnel
|   |   |   |-- ShareMapFSMBehaviour.java          # Protocole de partage
|   |   |   |-- ReceiveGolemTrailBehaviour.java    # Reception odeurs
|   |   |   |-- ReceiveSiegeStatusBehaviour.java   # Reception siege
|   |   |   |-- HuntBehaviour.java                 # (legacy / stand-alone)
|   |   |   |-- ExploCoopBehaviour.java            # (legacy)
|   |   |   '-- ...
|   |   '-- knowledge/
|   |       '-- MapRepresentation.java              # Carte locale + Dijkstra
|   '-- princ/
|       |-- Principal.java                          # Lanceur du conteneur
|       '-- ConfigurationFile.java
'-- src/test/java/...

```

L'architecture suit le principe : *un agent, plusieurs behaviours spécialisés, un état partagé local*. La classe `MapRepresentation` joue le rôle de *blackboard* interne à l'agent : elle est partagée entre tous les behaviours et constitue le seul point de mutation de la connaissance.

2.2 Diagramme des composants par agent

La figure 1 décrit l'organisation interne d'un `ExploreCoopAgent`. Quatre comportements y sont enregistrés en parallèle, partageant la même carte locale.

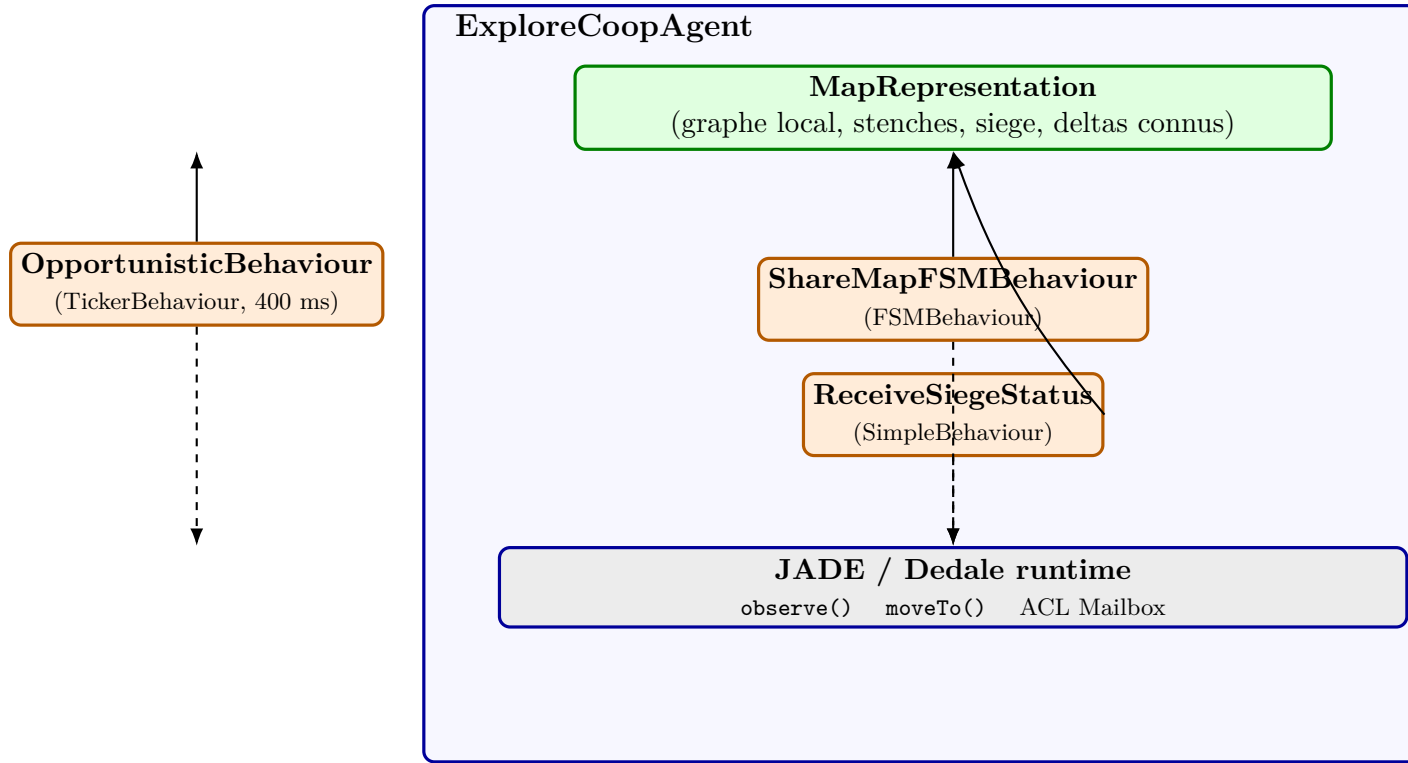


FIGURE 1 – Diagramme de composants d'un **ExploreCoopAgent**. Les flèches pleines représentent les accès en lecture/écriture sur la carte; les flèches pointillées les appels à l'environnement Dedale.

2.3 Diagramme d'états du comportement principal

Le comportement central **OpportunisticBehaviour** est un **TickerBehaviour** déclenché toutes les 400 ms. À chaque tick il sélectionne dynamiquement un mode parmi **EXPLORE** et **HUNT** selon les conditions de la figure 2.

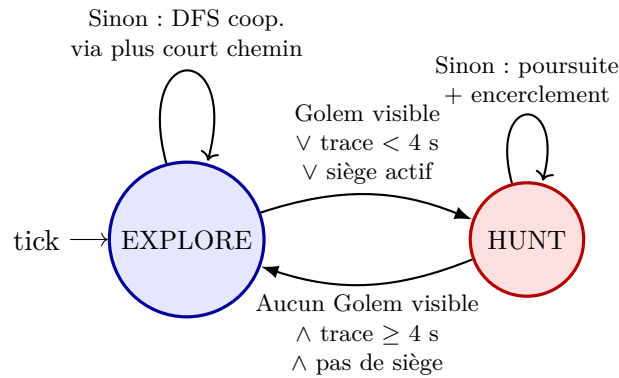


FIGURE 2 – Diagramme d'états du **OpportunisticBehaviour**. Le passage **EXPLORE**→**HUNT** est déclenché par tout indice frais indiquant la proximité d'un Golem; le retour s'effectue après expiration des traces.

2.4 Diagramme d'états du protocole de partage

Le **ShareMapFSMBehaviour** est une véritable FSM JADE à cinq états (figure 3).

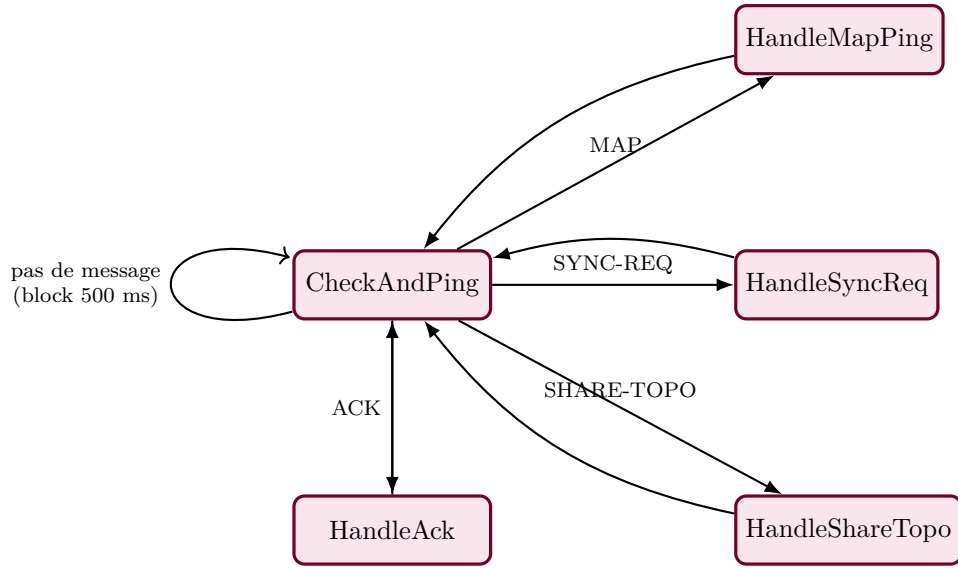


FIGURE 3 – FSM JADE du `ShareMapFSMBehaviour` : un état central de détection des messages, et quatre handlers spécialisés revenant à l’état initial.

2.5 Explication des interactions principales

Les interactions entre agents reposent sur cinq protocoles ACL identifiés par leur `protocol field` :

MAP	Ping périodique annonçant qu’un agent dispose d’une nouvelle version de carte. Émis toutes les 2 s, et uniquement si <code>updateCount</code> a augmenté ou si un agent est à portée visuelle.
SYNC-REQ	Réponse à un MAP : le récepteur demande explicitement le delta.
SHARE-TOPO	Envoi sérialisé d’un sous-graphe contenant uniquement les nœuds et arêtes que l’émetteur <i>sait</i> ne pas être déjà connus du destinataire (algorithme <code>getMapDelta</code>).
ACK	Confirmation de réception : permet à l’émetteur de marquer ce delta comme « connu du destinataire » (<code>markAsKnown</code>).
GOLEM_TRAIL	Diffusion (gossip non-fiable) d’une odeur fraîchement perçue : (<code>nodeId</code> , <code>stenchValue</code> , <code>timestamp</code>).
SIEGE_STATUS	État courant d’un siège : <code>golemPos</code> ; <code>golemName</code> ; <code>senderName</code> ; <code>senderPos</code> ; <code>holes</code> ; <code>timestamp</code> .
TARGET_CLAIM	Annonce, par un agent, du nœud qu’il s’apprête à occuper (utilisé pour résoudre les conflits de cible lors de l’encerclement).

Les trois derniers protocoles forment l’épine dorsale de la coordination de chasse, tandis que les quatre premiers gèrent la cohérence épistémique du graphe partagé.

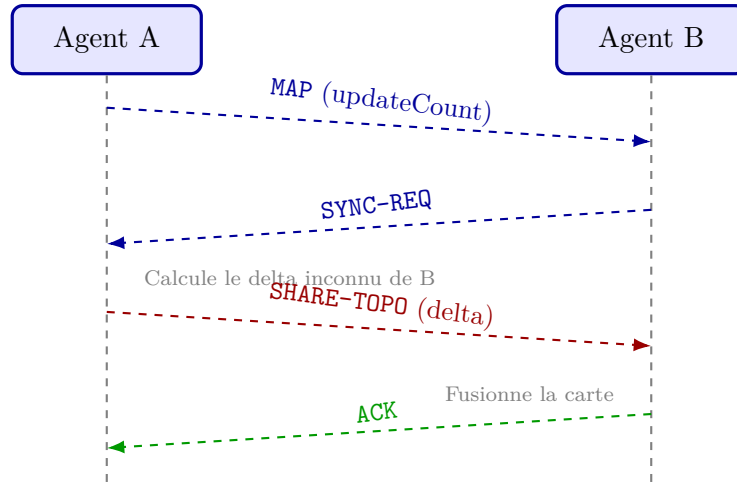


FIGURE 4 – Diagramme de séquence du protocole d'échange de carte avec delta et accusé de réception.

3 Choix d'algorithmes et stratégies retenues

3.1 Exploration

Stratégie retenue. Une exploration de type DFS-implicite par plus court chemin vers le *nœud ouvert* le plus proche (`getShortestPathToClosestOpenNode`). L'agent étiquette les nœuds visités comme `closed` et les voisins découverts mais non visités comme `open` (cf. `MapRepresentation.MapAttribute`).

Justification. Plutôt qu'un DFS récursif strict (qui imposerait de remonter le chemin), nous déléguons à Dijkstra le calcul du raccourci vers la frontière. Cela donne :

- un comportement **robuste à la perte d'information** (un agent peut « parachuter » via une fusion de carte sur un nœud inattendu et continuer de progresser) ;
- une **convergence naturelle vers les zones non explorées** sans backtracking explicite ;
- une **compatibilité directe** avec la structure `Graph` de `GraphStream` et son module `Dijkstra`.

Anti-collision. Si le premier nœud du chemin est occupé par un allié visible (`contientAgentsStrict`), un nœud accessible alternatif est sélectionné. Cela évite la formation de files indiennes caractéristique du `ExploCoopBehaviour` de référence.

Détection de fin d'exploration. `!myMap.hasOpenNode()` : aucun nœud `open` ne subsiste dans le graphe local. L'agent passe alors en mode *patrouille aléatoire* (random walk parmi les voisins non occupés), qui sert à la fois de maintenance de la carte et de détection passive des odeurs de Golem.

3.2 Communication

Protocole delta avec accusé de réception. Plutôt que d'envoyer la carte complète à chaque tick (comportement initial trop coûteux), nous diffusons uniquement les *deltas* non encore connus du destinataire, avec un mécanisme de mémoire de connaissance par pair : `knownNodesByAgent` et `knownEdgesByAgent`. Un delta n'est intégré dans cette mémoire qu'à la *réception d'un ACK*, garantissant que la perte d'un message ne provoque pas un trou de connaissance permanent.

Déclenchement adaptatif des pings. L’envoi d’un MAP ping n’est effectué que si :

$$(\text{maintenant} - t_{\text{lastPing}}) > 2 \text{ s} \quad \wedge \quad (\text{updateCount augmenté} \vee \text{allié visible}).$$

Cette double condition limite drastiquement le trafic réseau lorsque l’environnement est statique tout en garantissant la diffusion immédiate des nouvelles informations utiles.

Gossip pour les indices volatiles. Les GOLEM_TRAIL et SIEGE_STATUS ne sont *pas* acquittés : la donnée est volatile (timestampée et invalidée après quelques secondes), donc une perte ponctuelle est acceptable. Cela évite d’inonder le réseau avec des ACKs et reste cohérent avec le principe *eventually consistent*.

3.3 Coordination

Désambiguïsation des cibles par TARGET_CLAIM. Lorsque deux agents convergent vers un même nœud frontière (en exploration ou en chasse), ils diffusent leur intention sous la forme :

$$(\text{cible}, \text{timestamp}, \text{distance}, \text{score}).$$

Une fonction d’ordre lexicographique tranche localement et de manière identique pour tous : (1) plus petite distance, (2) plus grand `unattendedStenchesCount`, (3) ordre alphabétique du nom comme brise-égalité. Aucun consensus distribué n’est requis : le tri est déterministe et indépendant pour chaque agent à partir de la même information.

Encerclement par tri de degré. Pour le siège, lorsque le nombre d’agents disponibles est inférieur au degré du Golem ($A < D$), les « trous » du voisinage sont ordonnés par degré décroissant : on bouche en priorité les voisins qui mènent vers les zones les plus connectées (donc les plus dangereuses pour la fuite).

Résolution de *deadlock* par préséance lexicographique. Si un agent reste bloqué ≥ 3 ticks sur la même position et qu’un autre agent occupe sa cible, l’agent dont le nom est lexicographiquement *supérieur* cède la priorité (choix d’un nœud d’évasion aléatoire). Cette règle garantit l’absence d’interblocage symétrique sans synchronisation.

3.4 Chasse

Triangulation par intersection des voisinages. La fonction `getGolemPossibleLocations` calcule l’intersection des voisinages des nœuds émettant une odeur fraîche. Si cette intersection est un singleton, la position du Golem est déterminée avec certitude.

Dichotomie *Beater* / *Interceptor*.

- Le *rabatteur* (beater) : agent le plus proche de la trace la plus fraîche n_1 . Il avance vers n_1 par un voisin compatible avec la trajectoire estimée, en évitant le précédent nœud n_2 (déjà parcouru par le Golem).
- L’*intercepteur* : prédit la position future du Golem en propageant la trajectoire `predictGolemInterception` (jusqu’à 5 pas) en supposant qu’il fuit vers les nœuds de degré élevé. Il vise un nœud de coupe du périmètre.

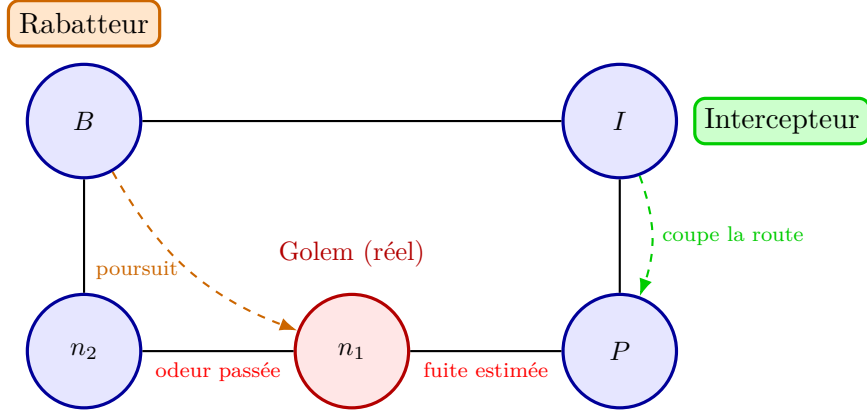


FIGURE 5 – Illustration de la stratégie de chasse : le rabatteur suit la piste la plus fraîche (n_1) depuis l’ancienne (n_2), tandis que l’intercepteur vise le point de fuite prédit (P).

Bascule EXPLORE/HUNT. Le passage en HUNT est conditionné par :

$$\text{golemVisible} \vee (\text{maxStench timestamp existe} \wedge \Delta t < 4 \text{ s}) \vee \text{siège actif}.$$

Le seuil de 4 s est calibré pour absorber la latence de propagation gossip tout en restant inférieur à la durée de vie d’une odeur (5 s).

4 Analyse des algorithmes

Pour chaque algorithme, on note V le nombre de nœuds du graphe, E le nombre d’arêtes, A le nombre d’agents alliés et S le nombre courant de nœuds porteurs d’une odeur.

4.1 Algorithme d’exploration

Forces.

- **Robustesse** aux fusions de carte (le DFS implicite redémarre depuis n’importe quel point cohérent).
- **Faible duplication** grâce à l’anti-collision sur les voisins immédiats.
- **Reprise gracieuse** après une phase de chasse : la classification `open/closed` reste valide.

Limites.

- Pas de **partition explicite** de l’espace entre agents (assignation par zone) : ils peuvent converger vers la même frontière si les anti-collisions locales sont insuffisantes.
- Recalcul d’un Dijkstra complet à chaque tick : pas de cache de plus court chemin.

Complexités.

- **Temps par tick** : $\mathcal{O}((V+E) \log V)$ (Dijkstra avec tas binaire) appelé une fois par `getShortestPathToClosest` plus $\mathcal{O}(|\text{open}|)$ pour la sélection du nœud cible.
- **Mémoire** : $\mathcal{O}(V+E)$ pour le graphe local, plus $\mathcal{O}(A \cdot V)$ pour les ensembles `knownNodesByAgent`.
- **Communication** : amortie en $\mathcal{O}(\Delta V + \Delta E)$ par cycle de partage (delta uniquement).

Optimalité. L’algorithme n’est **pas optimal au sens d’un coût de parcours minimum** (qui serait NP-difficile – problème de type *Watchman Route*). En revanche, il est *complet* : sous l’hypothèse d’un graphe connexe et d’au moins un échange de carte réussi entre tout couple d’agents, tout nœud accessible finit par être visité (ou découvert via fusion).

Critère d'arrêt. `!myMap.hasOpenNode()`. Une fois atteint, l'agent bascule en patrouille aléatoire et reste réactif aux événements de chasse (HUNT).

4.2 Algorithme de communication

Forces.

- **Économie de bande passante** : les deltas évitent la retransmission des nœuds connus.
- **Garantie de cohérence éventuelle** via les ACKs : un delta non confirmé reste à renvoyer tant qu'aucune trace de réception n'existe.
- **Découplage** entre carte topologique (avec ACK) et indices volatiles (gossip pur).

Limites.

- Mémoire $\mathcal{O}(A \cdot V)$ pour la table `knownNodesByAgent` : explose si A est grand.
- Si un message ACK est perdu (pas géré explicitement par Dedale, mais possible en cas de timeout), le delta peut être renvoyé à l'identique, doublant temporairement le trafic.
- Les protocoles `GOLEM_TRAIL` et `SIEGE_STATUS` sont best-effort : aucune garantie de livraison.

Complexités.

- **Temps par message reçu** : $\mathcal{O}(V + E)$ pour la fusion (`mergeMap`) ; $\mathcal{O}(V + E)$ pour le calcul de delta sortant.
- **Mémoire** : $\mathcal{O}(A(V + E))$.
- **Communication** : à l'état stationnaire, environ $\mathcal{O}(\Delta V + \Delta E)$ par paire d'agents et par cycle ; le pire cas (initial) est $\mathcal{O}(V + E)$ par paire.

Optimalité. Pas optimal au sens informationnel (le calcul du delta n'est pas un Set Cover) mais **quasi-optimal en pratique** : chaque arête est envoyée à chaque pair *au plus une fois* après réception d'ACK.

Critère d'arrêt. La FSM ne s'arrête *jamais* (boucle infinie sur `CheckAndPing`). Le critère pratique est l'absence de delta à émettre (`getMapDelta == null`) qui supprime silencieusement les envois.

4.3 Algorithme de coordination

Forces.

- **Décentralisation totale** : aucun leader, pas de consensus distribué requis.
- **Détermination locale identique** : tous les agents appliquent le même tri lexicographique sur les `TARGET_CLAIM`, donc convergent vers la même attribution sans négociation.
- Mécanisme *anti-deadlock* fondé sur le nom (totalement ordonné) : libération garantie en au plus 3 ticks.

Limites.

- Sensible à la **perte d'un TARGET_CLAIM** : deux agents peuvent croire être seuls sur leur cible.
- L'ordre lexicographique introduit un biais persistant : le même agent cède toujours en cas d'égalité parfaite.

Complexités.

- **Temps par tick** : $\mathcal{O}(D \cdot (V + E) \log V)$ où D est le degré du Golem (Dijkstra appelé une fois par trou).
- **Mémoire** : $\mathcal{O}(A)$ pour la table de `claimedTargets`, plus $\mathcal{O}(A)$ pour `siegeStaff`.
- **Communication** : $\mathcal{O}(A)$ messages par tick (un `TARGET_CLAIM` broadcast par agent).

Optimalité. Sous-optimal au sens combinatoire (l'allocation optimale agent→trou est un problème d'affectation $\mathcal{O}(n^3)$ par l'algorithme hongrois). L'heuristique gloutonne par distance + nom est *optimale localement* et **stable** (pas d'oscillation).

Critère d'arrêt. Implicite : la coordination cesse dès que le siège se dissout (Golem capturé, ou perte des odeurs pendant plus de 2 secondes : `cleanOldSiegeData`).

4.4 Algorithme de chasse

Forces.

- **Exploitation simultanée** de trois sources d'information : visibilité directe, triangulation par odeur, broadcast de siège.
- Stratégie *Beater / Interceptor* qui assure une couverture spatiale (un agent en tenaille, un autre en barrage).
- **Validation locale** du siège : si l'agent est en position et qu'il n'y a plus d'odeur ni de Golem visible, le siège est *réfuté* et la position effacée (`validateSiegeAndGetPos`). Ceci évite le siège fantôme.

Limites.

- La prédiction de fuite (depth-5) est gloutonne et peut être leurrée par un Golem à comportement non monotone.
- En cas d'odeur isolée (pas de second nœud n_2), la direction du Golem est inconnue et la stratégie d'interception se réduit à un encerclement aveugle.
- Coût en Dijkstra élevé : jusqu'à 5 appels par tick dans le pire cas (interception multi-profondeur).

Complexités.

- **Temps par tick** : $\mathcal{O}(S \cdot \text{deg}) + \mathcal{O}(k \cdot (V + E) \log V)$ avec $k \leq 5$ pour la triangulation et la prédiction.
- **Mémoire** : $\mathcal{O}(S)$ pour les **stenches** et leurs timestamps.
- **Communication** : $\mathcal{O}(A)$ broadcasts par tick (trail + claim + siege).

Optimalité. Non garantie. La capture optimale d'un agent mobile sur graphe (*Cops and Robbers*) est polynomiale uniquement sur certaines classes (graphes dits *cop-win*). Sur un graphe quelconque, aucune stratégie ne garantit une capture en temps fini avec un nombre fixe d'agents si le Golem est arbitrairement rapide. Notre algorithme est **capturant en pratique** sous les hypothèses : (i) $A \geq \text{cop-number}(G)$, (ii) le Golem ne dépasse pas la vitesse des explorateurs, (iii) le graphe est connexe.

Critère d'arrêt. Dans l'implémentation actuelle la chasse ne possède pas de critère d'arrêt explicite côté agent : c'est l'environnement Dedale qui retire le Golem à la capture, ce qui se traduit ensuite par la disparition des odeurs et des observations, faisant naturellement basculer l'agent vers EXPLORE.

5 Conclusion

5.1 Synthèse des résultats

Le système livré atteint les deux objectifs fixés :

1. Une **exploration coopérative complète** d'un environnement inconnu, robuste à la dissymétrie des cartes et à la perte ponctuelle de messages, grâce au protocole de partage delta avec accusé de réception.
2. Une **chasse coordonnée et opportuniste** qui combine triangulation olfactive, prédiction de trajectoire et siège distribué, sans aucune autorité centrale.

L'architecture retenue – un comportement décisionnel principal (`OpportunisticBehaviour`), une FSM dédiée au partage de carte (`ShareMapFSMBehaviour`) et deux récepteurs spécialisés – offre une séparation des responsabilités claire et un point d'entrée unique pour les évolutions futures.

5.2 Regard critique

Points forts.

- Le passage d'un comportement bicéphale (deux behaviours s'auto-tuant) à un `OpportunisticBehaviour` unifié a éliminé une classe entière de bugs liés au cycle de vie JADE.
- L'usage cohérent de l'ordre lexicographique sur le nom de l'agent comme tie-breaker fournit un mécanisme de désynchronisation léger et systémique.
- Le protocole delta est mesurable : la table `knownNodesByAgent` permettrait, sur une instrumentation future, de quantifier finement le surcoût communicationnel.

Points faibles.

- Aucune *auction* explicite : la coordination repose sur le tri local et peut se dégrader si la latence de `TARGET_CLAIM` dépasse celle d'un tick.
- Le calcul Dijkstra n'est pas mémoisé. Sur de grands graphes (centaines de nœuds), le coût par tick devient sensible.
- Le code conserve plusieurs *behaviours legacy* (`HuntBehaviour`, `ExploCoopBehaviour`) qui ne sont plus utilisés par `ExploreCoopAgent` mais restent dans l'arborescence : il subsiste un risque de confusion lors de la maintenance.

5.3 Extensions possibles

- **Partition de l'espace par Voronoï dynamique.** Assigner à chaque agent une région d'exploration prioritaire en fonction de sa position, recalculée à chaque fusion de carte. Cela réduirait encore la duplication d'effort.
- **Cache de plus court chemin.** Mettre en cache les distances « toutes paires » et invalider uniquement les entrées affectées par l'ajout d'un nœud/arête, pour passer de $\mathcal{O}((V+E) \log V)$ à $\mathcal{O}(1)$ amorti par requête.
- **Apprentissage du modèle de fuite du Golem.** Remplacer `predictGolemInterceptionPoint` (heuristique gloutonne) par un modèle de Markov estimé en ligne sur l'historique des odeurs.
- **Protocole d'auction explicite** (Contract-Net) pour les cibles de siège : un agent émet un *Call-For-Proposals*, les autres répondent avec leur distance, l'allocation est déterministe et auditable.
- **Fiabilisation du gossip** via numérotation de séquence : permettrait de réémettre les `GOLEM_TRAIL` manquants détectés par trou dans la séquence.

- **Visualisation centralisée** consolidée d’une carte « union », utile pour le debug mais sans dépendance fonctionnelle.
- **Nettoyage du code** : suppression des behaviours legacy et factorisation des fonctions `broadcast*/containsAlly` qui sont dupliquées entre `HuntBehaviour` et `OpportunisticBehaviour`.

Mot de la fin. Au-delà des résultats techniques, ce projet illustre un principe central des systèmes multi-agents : la coordination émergente issue de règles locales simples et déterministes peut rivaliser, en pratique, avec des architectures centralisées, tout en préservant la résilience et la modularité. L’*Opportunistic Behaviour* matérialise ce principe : il fait du même agent un explorateur quand le monde est calme, et un chasseur quand il s’éveille.